

MindCraft

Plateforme de Recherche Expérimentale

Documentation Technique — Version publique

Architecture, infrastructure, conformité et modèle de données

Version	1.2 — version publique (allégée)
Date	12 juin 2026
Plateforme	https://www.mindcraft-research.fr
Dépôt	https://github.com/mindcraft-research/mindcraft
Licence	GNU AGPL-3.0-or-later
DOI	10.5281/zenodo.19864887
SWHID	swh:1:rev:df8a8b127f4b7c024a140e56263d0e22927b33eb
ORCID auteure	0000-0002-4315-1058 (Dayle DAVID)
Contact	contact@mindcraft-research.fr

À propos de ce document

Cette documentation décrit l'architecture, l'implémentation et l'exploitation de MindCraft, plateforme open-source no-code pour la conception et la collecte d'expériences en ligne. Elle s'adresse aux équipes techniques (DSI, ESI, délégué·e·s à la protection des données, mainteneur·euse·s de logiciels de recherche) qui souhaitent évaluer la solution, l'auditer, l'intégrer dans un environnement institutionnel ou contribuer au code source.

Toutes les sections décrivent l'état réel du code en production à la date indiquée.

Portée de cette version publique

Ce document est une version allégée destinée à la publication ouverte. Les éléments opérationnels qui exposeraient inutilement la posture défensive de la plateforme ont été résumés sans valeurs précises. Sont notamment résumés sans détail : seuils précis de rate limiting et d'alerte, configuration interne de

l'authentification (rounds bcrypt, durées des tokens), bottlenecks identifiés, structure de la signature JWT, secrets CI/CD nominatifs, RTO/RPO chiffrés.

La version intégrale est mise à disposition sur demande motivée à contact@mindcraft-research.fr pour : DPO et délégué·e·s à la protection des données, équipes DSI/ESI dans le cadre d'une évaluation institutionnelle, rapporteur·euse·s scientifiques d'instances d'évaluation, auditeur·rice·s sécurité mandaté·e·s. Une réponse sous 5 jours ouvrés est garantie.

Sommaire

1. Présentation générale
2. Stack technique et justifications
3. Architecture applicative
4. Infrastructure et hébergement
5. Sécurité (résumé)
6. Pipeline CI/CD (résumé)
7. Conformité RGPD
8. Performance et scalabilité
9. Modèle de données
10. Monitoring et observabilité (résumé)
11. Open source et communauté
12. Moteur expérimental, stimuli et intégration physiologique
13. Algorithmes de contrebalancement et allocation
14. Annexe A : Référence des endpoints API
15. Annexe B : Variables d'environnement (résumé)
16. Annexe C : Glossaire

1. Présentation générale

MindCraft est une plateforme web no-code permettant à des chercheur·euse·s, doctorant·e·s, étudiant·e·s en master et praticien·ne·s des sciences humaines et comportementales de concevoir, déployer et collecter des données pour des protocoles expérimentaux en ligne. Elle vise à fournir, sans coût logiciel et sans dépendance commerciale, un outil rigoureux équivalent fonctionnellement aux solutions commerciales (Qualtrics, SurveyMonkey) tout en assurant la souveraineté des données et la transparence du code.

Le projet est né d'un constat simple : les outils dominants en sciences sociales sont soit chers et opaques (Qualtrics), soit puissants mais réservés à des programmeur·euse·s (jsPsych, lab.js, PsychoPy), soit datés et limités (LimeSurvey). Aucun ne combine simultanément (a) une interface no-code, (b) le support natif des tâches comportementales avec mesure de temps de réaction, (c) le contrebalancement expérimental automatique, (d) un hébergement souverain sous contrainte RGPD stricte, et (e) une licence libre audité. MindCraft adresse explicitement ces cinq axes.

1.1 Public cible technique

Cette documentation est destinée à quatre profils :

- DSI / Direction des Systèmes d'Information : évaluer la conformité sécurité et RGPD pour autoriser l'utilisation institutionnelle, éventuellement héberger une instance interne.
- Équipes système et réseau (ESI) : comprendre la topologie, les dépendances externes et les besoins réseau d'un déploiement.
- Délégué·e·s à la Protection des Données (DPO) : vérifier la conformité au RGPD article par article, identifier les sous-traitants et les bases légales.
- Mainteneur·euse·s de logiciels de recherche / contributeur·rice·s open-source : auditer le code, contribuer ou forker la plateforme pour des besoins spécifiques.

1.2 Choix architecturaux fondateurs

Cinq principes ont structuré les décisions techniques depuis la conception du projet :

- **No-code complet** : aucune ligne de code n'est requise pour créer une étude, même complexe (tâches expérimentales, designs factoriels, contrebalancement). Toute la logique est décrite via une interface graphique et sérialisée en JSON dans la base de données.
- **RGPD by design** : hébergement 100 % France, pseudonymisation par défaut des participant·e·s (UUID), consentement intégré dans le builder, suppression en cascade.
- **Open science par défaut** : métadonnées Open Science intégrées à chaque étude (préenregistrement, DOI, comités d'éthique), export du protocole en JSON pour reproductibilité, codebook PDF auto-généré, plateforme elle-même citable (DOI Zenodo, SWHID Software Heritage).
- **Souveraineté et réversibilité** : aucune dépendance à un cloud hyperscaler américain, données exportables à tout moment, code source publié sous AGPL pour permettre l'auto-hébergement institutionnel.
- **Reproductibilité scientifique** : timing des stimuli en sub-milliseconde via `performance.now()`, marqueurs LSL pour synchronisation EEG/ECG/eye-tracking, contrebalancement déterministe (Latin Square, Williams).

1.3 Comparaison succincte avec l'existant

Outil	No-code	RT/Tâche	Open source	RGPD/UE	Coût
MindCraft	Oui	Oui	AGPL-3.0	France	Gratuit
Qualtrics	Oui	Limité	Non	Variable	Élevé
LimeSurvey	Oui	Non	GPL	Auto-hébergeable	Gratuit
PsyToolkit	Partiel	Oui	Non (closed)	UE	Gratuit (académique)
jsPsych / lab.js	Non (code)	Oui	MIT	Auto-hébergeable	Gratuit
PsychoPy	Partiel	Oui	GPL	Auto-hébergeable	Gratuit

1.4 Ce que MindCraft n'est pas

Pour éviter toute confusion sur le périmètre fonctionnel :

- Pas un outil d'analyse statistique : les exports CSV/Excel/JSON sont destinés à R, Python (pandas), JASP, SPSS ou jamovi.
- Pas un panel de participant·e·s : la plateforme s'intègre avec Prolific, MTurk et SONA mais ne fournit pas elle-même de participant·e·s rémunéré·e·s.
- Pas un système d'acquisition physiologique : MindCraft émet des marqueurs LSL standardisés mais n'enregistre pas le signal brut EEG/ECG/eye-tracking, qui reste à la charge des logiciels d'acquisition (BrainVision, BIOPAC, Tobii, etc.).
- Pas un service commercial : même si le code est libre, l'instance hébergée à www.mindcraft-research.fr est strictement non-commerciale (recherche et enseignement uniquement).
- Pas une solution turnkey pour données particulièrement sensibles (données de santé au sens médical, données biométriques au sens article 9 RGPD) : ces usages nécessitent une analyse d'impact relative à la protection des données (AIPD) spécifique.

2. Stack technique et justifications

L'application est entièrement écrite en JavaScript (Node.js côté backend, React côté frontend), avec PostgreSQL comme base de données relationnelle et Prisma comme ORM type-safe. Toutes les briques sont open-source, activement maintenues, et leurs versions sont épinglées dans les fichiers package.json afin de garantir la reproductibilité des builds.

2.1 Synthèse des composants

Couche	Technologie	Version	Rôle
Frontend	Next.js + React	14.x / 18.x	Rendu hybride SSR/CSR, routing par fichier, SEO, hydratation cliente
State client	Zustand	4.x	Store global léger pour la session utilisateur·rice
Data fetching	TanStack Query	5.x	Cache intelligent, déduplication, retry, invalidation déclarative
Forms / valid.	JSON Schema (Fastify)	—	Validation des entrées au niveau API par schéma déclaratif
Styles	CSS Modules	—	Styles scopés par composant
Backend / API	Fastify	4.x	Framework HTTP performant, système de plugins
ORM	Prisma	5.x	Schéma déclaratif, migrations versionnées, queries type-safe
Base de données	PostgreSQL	16	SGBD relationnel ACID, JSONB, full-text search
Auth	JWT + bcrypt	—	Tokens signés, hash adaptatif des mots de passe
2FA	TOTP (RFC 6238)	—	Compatible Google Authenticator, Authy, 1Password
Email	Resend	—	API email transactionnel, datacenters UE
Export PDF	PDFKit	—	Génération côté serveur des codebooks et certificats
Export Excel	ExcelJS	—	Génération .xlsx multi-onglets
Images	Sharp (libvips)	—	Resize / optimisation des médias
Sanitization	sanitize-html / DOMPurify	—	Protection XSS, allowlist d'éléments et attributs
Containers	Docker	—	Build multi-stage
CI/CD	GitHub Actions	—	Tests, build, push registry, déploiement Scaleway
Tests	Vitest	4.x	Tests unitaires et intégration sur l'API backend

2.2 Justification des principaux choix

Node.js plutôt que Python ou PHP

Le choix de Node.js permet d'utiliser un langage unique côté frontend et backend (JavaScript), de partager des structures de données (validation, types) et de bénéficier de l'asynchrone non-bloquant nativement adapté à un trafic web. Python aurait également été viable (Django/FastAPI) mais aurait

introduit une dualité de langage. PHP a été écarté pour des raisons d'écosystème et de modernité (typage, ESM).

Fastify plutôt qu'Express

Fastify offre des performances brutes supérieures à Express, une validation des entrées intégrée via JSON Schema, une API moderne (async/await natif) et un écosystème de plugins officiels (cors, jwt, rate-limit, helmet, multipart, static). Express, bien que plus répandu, est moins typé, moins performant et impose des patterns plus verbeux.

Prisma plutôt que TypeORM, Sequelize ou Drizzle

Prisma fournit une expérience développeur-euse supérieure : schéma déclaratif unique (schema.prisma), migrations générées automatiquement et versionnées, queries fortement typées, query engine performant. Le compromis est une dépendance native qui nécessite OpenSSL en production (géré dans le Dockerfile).

PostgreSQL plutôt que MySQL ou MongoDB

PostgreSQL combine la rigueur relationnelle (transactions ACID, contraintes référentielles, intégrité) et la souplesse semi-structurée (type JSONB indexable). Cette dualité est essentielle pour MindCraft qui mêle des entités strictement relationnelles (User, Project, Study) et des configurations flexibles (questions par type, settings d'étude, métadonnées Open Science). MongoDB aurait perdu l'intégrité référentielle, MySQL aurait perdu la richesse JSONB et les fenêtres analytiques.

Next.js Pages Router plutôt qu'App Router

La plateforme utilise le Pages Router de Next.js 14. Ce choix est volontaire : le Pages Router est mature, stable, simple à raisonner (chaque fichier dans pages/ est une route), et son support du SSR pour le SEO reste plus éprouvé.

Scaleway plutôt qu'AWS, Azure ou GCP

Scaleway est un hébergeur français (siège à Paris), indépendant des hyperscalers américains, donc non soumis au CLOUD Act. Ses Serverless Containers offrent un modèle simple (push image Docker = redéploiement) avec auto-scaling natif et facturation à la seconde, particulièrement adapté à un service de recherche au trafic irrégulier.

3. Architecture applicative

3.1 Architecture trois-tiers

L'application suit une architecture trois-tiers classique (présentation, logique métier, persistance), chaque tier s'exécutant dans un container Docker indépendant et stateless déployé sur Scaleway Serverless Containers. Cette séparation permet une montée en charge horizontale par tier et un déploiement atomique avec rollback instantané.

Navigateur (Client) — HTTPS, navigateur moderne, JavaScript ES2022, React 18 →

Frontend (Next.js 14, Pages Router) — SSR/CSR hybride, Zustand, TanStack Query, www.mindcraft-research.fr →

API Backend (Fastify 4.x) — REST JSON, JWT, rate limiting, CORS strict, api.mindcraft-research.fr →

Base de données (PostgreSQL 16) — Prisma ORM, transactions ACID, TLS, Scaleway Managed (fr-par)

3.2 Architecture du frontend

Le frontend est organisé en cinq grands sous-systèmes, tous regroupés dans `frontend/src/` :

- **pages/** : routes Next.js (file-based routing). Pages statiques publiques, pages d'authentification, pages authentifiées, pages de projet et d'étude, portail participant public.
- **components/** : composants réutilisables. Layout, StaticLayout, CitationModal, FeedbackWidget. Sous-dossiers `builder/` (constructeur d'études), `runner/` (moteur de passation), `stimulus/` (moteur de tâches comportementales).
- **lib/** : services partagés (client API, store de session, source unique de vérité pour APA/BibTeX/RIS).
- **styles/** : globals CSS et thème.
- **middleware.js** : middleware Next.js exécuté sur toutes les requêtes (force HTTPS, headers de sécurité).

Stratégie de rendu

MindCraft combine SSR, SSG et CSR selon le type de page :

- Pages publiques marketing : SSG au build (HTML statique servi à l'edge, optimal pour le SEO).
- Portail participant : SSR à chaque requête, pour permettre une indexation correcte et un premier affichage rapide.
- Pages authentifiées : CSR après une coquille SSR minimale.

3.3 Architecture du backend

Le backend Fastify est organisé en plugins enregistrés dans `backend/src/server.js` : CORS, cookie, JWT, rate-limit, helmet, multipart, static, ainsi qu'un plugin maison qui injecte un client Prisma singleton.

Les routes sont réparties par domaine fonctionnel dans `backend/src/routes/` :

- `auth.js`, `projects.js`, `studies.js`, `design.js`, `run.js` (passation publique), `export.js`, `admin.js`, `media.js`, `stimulus.js`, `feedback.js`.

3.4 Modèle de communication

La communication frontend/backend repose exclusivement sur HTTP/JSON suivant les conventions REST. Pas de GraphQL, pas de WebSocket côté application (sauf pour l'intégration LSL via un relay externe). Verbes HTTP standards, codes de retour HTTP standards, authentification par token JWT, validation par JSON Schema, pagination conventionnelle.

4. Infrastructure et hébergement

Données hébergées en France

Toute l'infrastructure (containers, base de données, registre d'images Docker) est hébergée chez Scaleway, datacenter Paris (région fr-par). Le nom de domaine est géré chez OVH (Roubaix). Aucune donnée personnelle ou expérimentale ne transite hors UE. Les emails transactionnels sont sous-traités à Resend, dont les serveurs sont également en UE.

4.1 Composants Scaleway (résumé)

Composant	Service Scaleway	Fonction
Frontend	Serverless Container	Auto-scaling, HTTPS natif, custom domain
Backend API	Serverless Container	Auto-scaling, HTTPS natif, env vars chiffrées
Base de données	Managed PostgreSQL	PostgreSQL 16, SSL/TLS imposé, sauvegardes automatiques
Container Registry	Container Registry	Registre privé fr-par.scw.cloud, rétention configurable

Note : les paramètres précis d'auto-scaling (min_scale, max_scale, max_concurrency, limites mémoire/CPU) et les objectifs de reprise (RTO/RPO) sont documentés dans la version intégrale réservée aux évaluateurs·rice·s.

4.2 Résolution DNS et certificats

Les domaines apex et www sont publiés sur OVH, les certificats TLS sont émis automatiquement par Let's Encrypt via Scaleway et renouvelés tous les 90 jours sans intervention. La négociation TLS impose au minimum TLS 1.2 (TLS 1.3 négocié en priorité). HSTS activé avec includeSubDomains et preload.

4.3 Gestion des variables d'environnement et secrets

Aucun secret n'est stocké dans le code source. Trois zones de configuration coexistent : développement local (.env git-ignore), CI/CD GitHub Actions (secrets chiffrés au repos), production Scaleway (variables d'environnement chiffrées au repos). Le résumé des variables figure à l'Annexe B.

4.4 Sauvegardes et plan de reprise

Trois niveaux de sauvegarde se complètent : code source (redondance Git GitHub + archivage Software Heritage), images Docker (Container Registry Scaleway, tag par release pour rollback), base de données (snapshots automatiques quotidiens + point-in-time recovery). Les objectifs RTO/RPO précis sont documentés dans la version intégrale.

4.5 Topologie réseau

La base de données PostgreSQL n'est pas exposée directement sur Internet : elle est accessible uniquement par le container backend via une URL privée Scaleway et une connexion TLS. Les containers frontend et backend exposent leurs ports via le routeur Scaleway qui termine TLS et applique l'auto-scaling.

5. Sécurité (résumé public)

Version résumée

Cette section présente la posture de sécurité à un niveau permettant l'évaluation externe sans détailler les paramètres opérationnels (seuils de rate limiting, durées exactes des tokens, structure de signature, etc.). Le détail technique est documenté dans la version intégrale, mise à disposition sur demande motivée des DPO, DSI, ESI, rapporteur·euse·s scientifiques et auditeur·rice·s sécurité.

5.1 Modèle de menace

Trois catégories d'adversaires sont considérées :

- **Attaquant·e·s externes anonymes** : tentent de compromettre les comptes par brute-force, d'injecter du code (XSS, SQL injection), de dévier le trafic (MITM, DNS hijacking), d'extraire des données via des routes mal protégées.
- **Utilisateur·rice·s authentifié·e·s malveillant·e·s** : tentent d'accéder aux données d'autres chercheur·euse·s (escalade horizontale), d'obtenir des privilèges admin (escalade verticale), de saturer le service (DoS).
- **Adversaires internes potentiel·le·s** : un·e administrateur·rice Scaleway, un·e mainteneur·euse du code, un·e sous-traitant·e. La confidentialité côté infrastructure repose sur les engagements contractuels de Scaleway (DPA, certifications ISO 27001, SOC 2).

5.2 Mécanismes d'authentification

La plateforme implémente :

- Hachage des mots de passe avec bcrypt à coût adapté.
- Politique de complexité minimale des mots de passe vérifiée côté serveur.
- Tokens JWT signés (access court + refresh long, rotation automatique).
- Refresh token stocké dans un cookie HttpOnly Secure SameSite=Strict.
- Double authentification optionnelle par TOTP (RFC 6238), compatible Google Authenticator, Authy, 1Password.
- Vérification d'email à l'inscription par token expirant signé.
- Réinitialisation de mot de passe par token hashé en base, anti-replay.
- Système de rôles USER/ADMIN avec middleware route-level.

5.3 Autorisation

Trois niveaux d'autorisation se superposent sur chaque route : authentification (token valide), rôle (ADMIN si requis), ressource (propriétaire ou collaborateur·rice). La logique est centralisée dans des fonctions helper appelées au début de chaque handler.

5.4 Protections contre les attaques

Menace	Protection
XSS	sanitize-html côté backend + DOMPurify côté frontend

CSRF	Cookies SameSite=Strict + whitelist CORS stricte
Injection SQL	ORM Prisma avec queries paramétrées exclusivement
Brute-force	@fastify/rate-limit par IP et par route
Headers sécurité	@fastify/helmet (HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy)
Force HTTPS	Middleware Next.js (redirect + HSTS preload)
MITM	TLS 1.2+ obligatoire, SSL sur la BDD
Path traversal	Hash des noms de fichiers uploadés, lookup par ID
Upload malveillant	Allowlist MIME, vérification magic bytes, SVG/HTML bloqués

Note : les seuils précis de rate limiting par route et les paramètres détaillés (durées de validité des tokens, fenêtres de tolérance, structure exacte de la signature JWT) sont documentés dans la version intégrale.

5.5 Audit et traçabilité

Chaque action sensible (création/suppression d'étude, invitation de collaborateur·rice, modification de mot de passe, activation/désactivation 2FA, suppression de compte) est journalisée dans une table dédiée avec userId, action, métadonnées JSON et timestamp. Cette table est consultable par les administrateur·rice·s et permet de reconstituer l'historique d'un compte en cas d'incident.

5.6 Gestion des secrets

Aucun secret n'est versionné dans le code source. La gestion s'appuie sur les mécanismes natifs des plateformes (GitHub Encrypted Secrets en CI/CD, variables d'environnement chiffrées au repos chez Scaleway en production, .env local git-ignore en développement). Un audit anti-secrets a été effectué avant le passage en open-source sur tout l'historique git.

6. Pipeline CI/CD (résumé public)

Le pipeline est défini dans `.github/workflows/ci.yml` et déclenché par GitHub Actions sur chaque push vers main et sur chaque pull request. Il enchaîne quatre jobs séquentiels et conditionnels :

- **backend-test** : installation des dépendances, génération Prisma, démarrage smoke test, exécution des tests Vitest.
- **frontend-build** : build Next.js avec injection de l'URL d'API au build.
- **build-images (push main)** : build Docker multi-stage, push vers le Container Registry Scaleway.
- **deploy (push main)** : redéploiement des containers Scaleway et attente de la sonde de santé.

6.1 Stratégie Docker multi-stage

Les Dockerfiles (backend et frontend) suivent un pattern multi-stage : stage builder (outils de compilation + génération Prisma + build), stage production (copie sélective vers une image légère). Image de base Debian slim pour la compatibilité avec les modules natifs. Container exécuté en non-root.

6.2 Branch protection

La branche main est protégée par un ruleset GitHub : pull request obligatoire avant merge, dismiss stale approvals on new commits, require conversation resolution, block force pushes, block deletions. Le nombre d'approbations requises sera relevé progressivement avec la croissance de l'équipe de mainteneur·euse·s.

6.3 Maintenance automatisée des dépendances

Dependabot ouvre hebdomadairement des Pull Requests pour mettre à jour les dépendances npm (backend + frontend), GitHub Actions et images Docker de base. Une analyse statique sécurité (SAST) supplémentaire est planifiée.

6.4 Stratégie de release

Les releases suivent une variante allégée de SemVer adaptée à un projet de recherche académique : v0.x.y (alpha académique, phase actuelle) puis v1.x.y (stabilisé). Chaque release publique déclenche automatiquement, via l'intégration GitHub-Zenodo, l'archivage de l'état du dépôt et la frappe d'un nouveau DOI.

7. Conformité RGPD

Hébergement 100% France / UE

Toutes les données personnelles et expérimentales sont stockées chez Scaleway (DC Paris). Aucun sous-traitant hors UE n'est utilisé pour le stockage ou le traitement. Le sous-traitant email Resend opère depuis des datacenters UE. La plateforme s'engage à maintenir cette localisation sur la durée de vie du projet.

7.1 Mapping des exigences RGPD

Mesure	Implémentation	Référence
Localisation données	France (Scaleway DC Paris, OVH Roubaix)	Conformité UE
Minimisation	Seules les données nécessaires aux finalités déclarées	Art. 5.1.c
Limitation finalité	Données de recherche uniquement, pas de marketing	Art. 5.1.b
Chiffrement en transit	HTTPS/TLS partout, SSL imposé sur BDD	Art. 32
Chiffrement au repos	BDD Scaleway (AES-256), backups chiffrés	Art. 32
Pseudonymisation	Participant·e·s identifié·e·s par UUID anonyme	Art. 25
Mots de passe	Hachage bcrypt, irréversible	Art. 32
Droit d'accès	Export JSON personnel	Art. 15
Droit à la portabilité	Export CSV / Excel / JSON	Art. 20
Droit de suppression	Suppression compte avec cascade complète	Art. 17
Droit de rectification	Modification profil et settings depuis l'UI	Art. 16
Consentement	Bloc CONSENT du builder, opt-in explicite	Art. 7
Information	Politique de confidentialité, CGU, mention CONSENT	Art. 13
Journalisation	Table ActivityLog	Art. 5.2
Sous-traitant email	Resend (Delaware US mais infra UE) — DPA signé	Art. 28
Sous-traitant hébergement	Scaleway SAS (France) — DPA signé	Art. 28
Sous-traitant DNS	OVH (France) — DPA signé	Art. 28
Notification de violation	Procédure interne, notification CNIL sous 72h	Art. 33
Licence du code	AGPL-3.0-or-later, code public auditable	Open science / Art. 32

7.2 Sous-traitants (chaîne de traitement)

Sous-traitant	Rôle	Localisation	DPA
Scaleway SAS	Hébergement (containers, BDD, registry)	Paris, France	Signé
OVH SAS	DNS, registrar	Roubaix, France	Signé (CGV)

Resend Inc.	Emails transactionnels (vérification, reset)	Datacenters UE	Signé
Zenodo (CERN)	Archivage de releases publiques (code seulement)	Genève, Suisse	CGU
Software Heritage	Archivage du code source (public)	Inria, France	CGU
GitHub (Microsoft)	Hébergement code source public	US (code public)	CGU

Important : aucun sous-traitant ne traite des données de participant·e·s (données expérimentales). Resend ne traite que des adresses email de chercheur·euse·s. Zenodo, Software Heritage et GitHub n'archivent que du code source public.

7.3 Données collectées et finalités

Données sur les chercheur·euse·s (utilisateur·rice·s inscrit·e·s)

- **Identifiants** : nom d'utilisateur·rice, adresse email (vérifiée).
- **Authentification** : hash bcrypt du mot de passe, secret 2FA chiffré (optionnel).
- **Profil** : prénom, nom, institution (optionnels, déclaratifs).
- **Activité** : journaux d'action.
- **Base légale** : exécution du contrat (CGU) et intérêt légitime (sécurité, prévention des fraudes).

Données sur les participant·e·s (anonymes)

- **Identifiant pseudonyme** : UUID v4 généré aléatoirement.
- **Réponses aux questions** : valeurs brutes, type-dépendantes.
- **Données comportementales** : temps de réaction par essai, touche pressée, exactitude.
- **Données techniques** : timestamps haute précision, identifiants de bloc, condition expérimentale.
- **Données de recrutement (le cas échéant)** : PROLIFIC_PID, MTurk workerId, source de recrutement.
- Aucun nom, prénom, email, IP du·de la participant·e n'est collecté par défaut. Le·la chercheur·euse peut ajouter ces champs s'il y a une base légale (consentement explicite via bloc CONSENT).

7.4 Workflow des droits des personnes

Pour les chercheur·euse·s : accès (export JSON depuis settings), suppression (cascade complète depuis settings, irréversible), rectification (modification du profil depuis l'UI).

Pour les participant·e·s : la plateforme ne gère pas directement les droits des participant·e·s car elle ne dispose pas de leurs identifiants directs (pseudonymisation). Les participant·e·s doivent s'adresser au·à la chercheur·euse responsable de l'étude.

8. Performance et scalabilité

8.1 Choix techniques au service de la performance

Technologie	Avantage performance	Couche
Fastify	Framework Node.js performant, validation intégrée	Backend
Prisma	Query engine compilé en Rust, connection pooling	ORM
TanStack Query	Cache client intelligent, déduplication	Frontend
Next.js SSR	Premier affichage rapide côté serveur	Frontend
Sharp	Traitement images natif via libvips (C++)	Média
Serverless	Auto-scaling selon la charge	Infrastructure
CDN edge	Assets statiques cachés en bordure de réseau	Réseau
HTTP/2	Multiplexing des requêtes	Transport

8.2 Stratégie de cache

- Frontend : TanStack Query avec stale-while-revalidate sur la plupart des queries.
- Pages statiques Next.js : SSG au build, servies depuis le CDN edge.
- Backend : pas de cache externe (Redis/Memcached). Prisma utilise du connection pooling.
- Base de données : PostgreSQL gère ses pages chaudes via shared_buffers.

8.3 Indexes critiques en base

Des indexes sont déclarés dans le schéma Prisma sur les champs accédés en chemin chaud (identifiants des entités principales, clés étrangères pour les jointures, colonnes utilisées pour le tri ou le filtrage des sessions de participation, des réponses et des journaux d'activité).

8.4 Auto-scaling et limites connues

L'architecture serverless permet une montée en charge automatique sur le frontend et le backend. La base de données peut être scalée verticalement à la demande. Les containers sont stateless, ce qui permet un scaling horizontal sans contraintes de session.

Note : le détail des paramètres d'auto-scaling et la liste exhaustive des bottlenecks identifiés sont documentés dans la version intégrale, pour ne pas exposer publiquement la posture défensive de la plateforme.

8.5 Stratégie de scalabilité horizontale

- Containers stateless : aucune session locale, chaque requête peut être traitée par n'importe quelle instance.
- Base de données managée en haute disponibilité.
- Transactions sérialisables pour l'allocation des participant·e·s : évite les conditions de course quand plusieurs participant·e·s arrivent simultanément.
- Assets statiques distribuables via CDN.

9. Modèle de données

Le schéma est géré par Prisma ORM, déclaré dans backend/prisma/schema.prisma. Toutes les modifications passent par des migrations versionnées. La base de données PostgreSQL 16 est hébergée chez Scaleway (Managed PostgreSQL).

Note : le schéma Prisma complet est public dans le dépôt à backend/prisma/schema.prisma. Cette section en présente la hiérarchie d'entités à un niveau synthétique.

9.1 Hiérarchie des entités principales

Entité	Description	Cardinalité
User	Compte chercheur·euse. Authentification, profil, rôle, 2FA	Racine
Project	Conteneur logique. Possédé ou partagé avec d'autres User	1 User → N
ProjectCollaborator	Lien User-Project avec rôle (EDITOR/VIEWER)	1 Project → N
Study	Expérience. Statut DRAFT → COLLECTING → ARCHIVED	1 Project → N
Block	Bloc d'étude (WELCOME / INSTRUCTION / QUESTION / STIMULUS / LOGIC / DEBRIEFING / TASK)	1 Study → N
Question	35+ types, choices, matrix items, conditions d'affichage	1 Block → N
ExperimentalDesign	Facteurs, niveaux, type de contrebalancement	1 Study → 0..1
ParticipantSession	Une passation. Statut, allocation, ordre des blocs	1 Study → N
QuestionResponse	Réponse d'un·e participant·e à une question	1 Session → N
StimulusResponse	Réponse d'un essai (RT, touche, exactitude)	1 Session → N
MediaFile	Fichier média uploadé par le·la chercheur·euse	1 User → N
ActivityLog	Action journalisée (audit trail)	1 User → N
RefreshToken	Token de session active	1 User → N

9.2 Types de questions (35+)

Catégorie	Types disponibles
Choix unique	Radio, Select, Button Group, Media Radio, Drill-down
Choix multiple	Checkbox, Media Checkbox, Checkbox + commentaire libre
Texte	Texte libre (court / long), Saisie structurée, Texte à trous
Échelles	Likert (3-11 points), Matrice, Slider, Différentiel sémantique, Somme constante
Numérique	Saisie numérique (bornes), Équation, Champ calculé
Interactif	Classement, Drag-and-drop libre, Hotspot, Surlignage de texte
Média	Image, Audio, Vidéo
Spécial	Consentement (CONSENT), Date, Upload, Timing, Meta-info

10. Monitoring et observabilité (résumé)

10.1 Composants

Composant	Outil	Description
Logs applicatifs	Pino (Fastify)	Logs structurés JSON, niveaux info/warn/error/fatal
Logs containers	Scaleway Cockpit	Centralisation logs Docker
Métriques infra	Scaleway Cockpit	CPU, RAM, requêtes/s, latence, code de retour
Métriques BDD	Scaleway Cockpit	Connexions, queries/s, taille, replication lag
Health check	GET /api/health	Vérifie API + connexion BDD, retourne version
Activity logs	Table ActivityLog	Traçabilité des actions utilisateur·rice (audit)
Alertes	Scaleway Alerting	Notifications email sur seuils critiques

10.2 Health check public

La route GET /api/health retourne un JSON minimal (statut, version, uptime, état de la BDD, timestamp). Elle sert de sonde de santé à Scaleway et à tout outil de monitoring externe. En cas de problème détecté, la route retourne 503 et Scaleway redirige le trafic vers une autre instance.

10.3 Seuils d'alerte

Des seuils d'alerte sont configurés sur le taux d'erreurs 5xx, la latence p95, l'usage CPU/mémoire, les connexions à la base de données et le remplissage disque. Les valeurs précises sont documentées dans la version intégrale pour éviter d'exposer les fenêtres exploitables par un·e attaquant·e.

11. Open source et communauté

Le code source de MindCraft est publié sur GitHub sous licence GNU AGPL-3.0-or-later. Cette ouverture garantit la reproductibilité scientifique, la vérifiabilité des traitements de données, et permet à la communauté de recherche de contribuer, auditer ou dériver la plateforme. La première version publique (v0.1.0) a été archivée sur Zenodo et Software Heritage.

Portée de la licence

L'AGPL-3.0-or-later s'applique uniquement au code source publié sur GitHub. Le service hébergé à <https://www.mindcraft-research.fr> reste régi par ses propres Conditions Générales d'Utilisation, qui imposent un usage non-commercial dans le cadre de la recherche académique. Un fork commercial doit (1) republier son code dérivé sous AGPL et (2) ne pas réutiliser la marque MindCraft.

11.1 Identifiants pérennes

Type	Valeur	Usage
DOI Zenodo	10.5281/zenodo.19864887	Citation académique, frappe à chaque release
SWHID	swh:1:rev:df8a8b127f4b7c024a140e56263d0e22927b33eb	Archive pérenne du code source
ORCID auteure	0000-0002-4315-1058	Identifiant chercheur (Dayle DAVID)
Repository	github.com/mindcraft-research/mindcraft	Code source ouvert AGPL-3.0-or-later

11.2 Fichiers de gouvernance du projet

- **LICENSE** : texte intégral de la GNU AGPL-3.0 + note de portée bilingue.
- **README.md** : présentation, installation locale, badges (license, DOI, live).
- **CONTRIBUTING.md** : procédure de contribution (fork, PR, tests, branch protection).
- **CODE_OF_CONDUCT.md** : Contributor Covenant 2.1 adapté.
- **SECURITY.md** : politique de divulgation responsable des failles.
- **codemeta.json** : métadonnées JSON-LD (schéma codemeta v2.0).
- **CITATION.cff** : format de citation, bouton « Cite this repository » GitHub.
- **.github/ISSUE_TEMPLATE/** : templates bug_report et feature_request structurés.
- **.github/PULL_REQUEST_TEMPLATE.md** : checklist de PR (tests, doc, AGPL acceptance).
- **.github/dependabot.yml** : mises à jour automatiques hebdomadaires.

11.3 Traçabilité scientifique

- Chaque modification du code passe par une Pull Request reviewée (branch protection ruleset).
- Headers SPDX-License-Identifier: AGPL-3.0-or-later sur les fichiers source principaux.
- Identifiants chercheur ORCID inclus dans codemeta.json et CITATION.cff.

- Archive Zenodo (DOI) et Software Heritage (SWHID) pour chaque release publique.
- Historique git complet et auditable depuis l'origine du projet.
- Citation automatique depuis l'application : fenêtre proposant les formats APA, BibTeX et RIS.

12. Moteur expérimental, stimuli et intégration physiologique

Cette section décrit le fonctionnement du moteur de présentation des stimuli et la chaîne de mesure du temps de réaction, points critiques pour la validité scientifique des tâches comportementales en ligne. Elle couvre également l'intégration avec les équipements physiologiques via Lab Streaming Layer (LSL).

12.1 Architecture du moteur de stimuli

Le moteur de stimuli est implémenté côté frontend dans `frontend/src/components/stimulus/`. Il s'exécute exclusivement dans le navigateur du participant pour minimiser la latence réseau et garantir la précision temporelle. Le serveur intervient uniquement en début et fin de bloc (chargement de la configuration, persistance des réponses).

12.2 Chaîne temporelle d'un essai

Un essai standard suit la séquence suivante : croix de fixation, stimulus, capture de la réponse, affichage feedback optionnel, inter-trial interval. Tous les médias sont préchargés en mémoire avant le bloc pour éliminer la latence réseau pendant les essais. Les timestamps de présentation effective sont capturés via `performance.now()` au moment du paint suivant (`requestAnimationFrame`).

12.3 Précision temporelle

La précision temporelle repose sur trois mécanismes complémentaires :

- **`performance.now()`** : horloge monotonique du navigateur, résolution typique de 5 à 100 microsecondes selon le navigateur. Insensible aux ajustements d'horloge système (NTP).
- **`requestAnimationFrame`** : synchronisation avec le cycle de rafraîchissement de l'écran (typiquement 60 Hz).
- **Pre-loading des médias** : chaque image, audio ou vidéo est entièrement chargé en mémoire avant le début du bloc.

La littérature établit que le RT mesuré dans un navigateur web est généralement de l'ordre de 5 à 30 ms plus long que dans un environnement natif (PsychoPy desktop), avec un bruit additionnel de 5 à 15 ms (écart-type). Cette latence systémique est constante et donc compensable dans les analyses, sauf pour les paradigmes à haute exigence temporelle (sub-millisecondes type masquage perceptif), qui restent l'apanage des environnements desktop.

Limitation des études en ligne

Pour des paradigmes exigeant une précision temporelle sub-milliseconde absolue (par ex. masquage visuel, P300 EEG synchronisé au stimulus), une expérimentation en laboratoire avec PsychoPy ou jsPsych en mode kiosque est préférable. MindCraft est calibré pour la majorité des paradigmes en sciences sociales (RT typique 200 à 2000 ms) où la latence systémique reste négligeable face à la variance interindividuelle.

12.4 Intégration physiologique LSL

Lab Streaming Layer (LSL) est un protocole standard en neurosciences pour synchroniser plusieurs flux de données (EEG, eye-tracking, GSR, vidéo) sur une horloge commune. MindCraft envoie des

marqueurs événementiels qui sont ensuite combinés aux signaux acquis par les logiciels d'acquisition (BrainVision, BIOPAC, Tobii Pro Lab, OpenBCI, etc.).

MindCraft émet deux niveaux de marqueurs : marqueurs globaux (niveau étude : début/fin étude et bloc, affichage et réponse question) et marqueurs fins (niveau tâche : fixation, stimulus, réponse, feedback). Le relay LSL est un script Python externe à lancer côté chercheur-euse.

13. Algorithmes de contrebalancement et allocation

Le contrebalancement est essentiel en sciences expérimentales pour contrôler les effets d'ordre (fatigue, apprentissage, contamination). MindCraft implémente trois algorithmes déterministes ainsi qu'un mécanisme d'allocation atomique des participant·e·s utilisant les transactions sérialisables PostgreSQL.

13.1 Algorithmes implémentés

Latin Square (carré latin)

Pour un facteur à N niveaux, génère une matrice NxN où chaque niveau apparaît exactement une fois par ligne et par colonne. La k-ième ligne donne l'ordre de présentation pour le·la k-ième participant·e. Garantit l'équilibre du premier ordre (chaque niveau est en première position pour exactement un·e participant·e sur N).

Williams Design

Variante du carré latin qui équilibre également les effets de premier ordre carryover (effet résiduel d'un niveau sur le suivant). Pour N pair, une seule matrice NxN suffit ; pour N impair, deux matrices sont nécessaires (2N participant·e·s par cycle).

Randomisation simple

Pour chaque participant·e, le moteur tire un ordre aléatoire des niveaux par mélange de Fisher-Yates. Pas d'équilibre garanti sur les premier·e·s participant·e·s mais converge vers l'équilibre asymptotiquement. Recommandé quand N est grand (>10 participant·e·s par cellule).

13.2 Allocation atomique des participant·e·s

Quand un·e participant·e arrive sur une étude avec quotas stricts, il·elle doit être alloué·e à une cellule expérimentale. Plusieurs participant·e·s pouvant arriver simultanément, l'allocation doit être atomique. MindCraft utilise une transaction PostgreSQL en isolation Serializable : sélection de la cellule la moins remplie, incrémentation du compteur, création de la session, le tout dans une transaction unique. En cas de conflit, PostgreSQL avorte l'une des transactions, automatiquement retentée par le code.

13.3 Quotas stricts vs souples

- **Strict** : une fois la cellule remplie, aucun·e nouveau·elle participant·e n'y est alloué·e. Si toutes les cellules sont pleines, l'étude est marquée comme complète.
- **Souple** : la cellule est privilégiée jusqu'à `target_count`, mais des participant·e·s supplémentaires peuvent être accepté·e·s au-delà. Utile pour amortir les abandons.

13.4 Déterminisme et reproductibilité

Tous les algorithmes sont déterministes à graine fixée. La graine de randomisation est stockée dans `ExperimentalDesign.seed`. Réexécuter le contrebalancement avec la même graine produit les mêmes affectations, condition essentielle pour la reproductibilité scientifique en cas de ré-analyse ou d'audit.

14. Annexe A : Référence des endpoints API

Liste exhaustive des endpoints publics et authentifiés. La colonne Auth indique le niveau d'autorisation requis : Public (aucune), User (token JWT valide), Owner (propriétaire de la ressource), Admin (rôle=ADMIN).

14.1 Authentification

Méthode	Route	Auth	Description
POST	/api/auth/register	Public	Inscription, envoi email de vérification
POST	/api/auth/verify-email	Public	Confirmation via token
POST	/api/auth/resend-verification	Public	Renvoi de l'email de vérification
POST	/api/auth/login	Public	Connexion, retourne access + cookie refresh
POST	/api/auth/refresh	Public (cookie)	Rotation refresh token
POST	/api/auth/logout	User	Invalidation du refresh token
GET	/api/auth/me	User	Profil de l'utilisateur·rice connecté·e
POST	/api/auth/forgot-password	Public	Envoi email de reset
POST	/api/auth/reset-password	Public	Reset effectif via token
POST	/api/auth/2fa/enroll	User	Génération secret TOTP, QR code
POST	/api/auth/2fa/verify	User	Activation 2FA après validation du code
POST	/api/auth/2fa/login-verify	Public (temp)	Validation 2FA en flux de connexion
DELETE	/api/auth/account	User	Suppression du compte avec cascade
GET	/api/auth/export	User	Export RGPD des données personnelles

14.2 Projets et collaborations

Méthode	Route	Auth	Description
GET	/api/projects	User	Liste des projets
POST	/api/projects	User	Création d'un projet
GET	/api/projects/:id	User+access	Détail d'un projet
PATCH	/api/projects/:id	Owner	Mise à jour du projet
DELETE	/api/projects/:id	Owner	Suppression du projet (cascade)

POST	/api/projects/:id/invite	Owner	Invitation collaborateur·rice
POST	/api/projects/:id/accept	User	Acceptation d'invitation
DELETE	/api/projects/:id/collab/:userId	Owner	Retrait d'un·e collaborateur·rice

14.3 Études, blocs, questions

Méthode	Route	Auth	Description
GET	/api/studies/:id	User+access	Détail d'une étude
POST	/api/projects/:id/studies	Editor	Création d'étude
PATCH	/api/studies/:id	Editor	Mise à jour métadonnées étude
DELETE	/api/studies/:id	Owner	Suppression étude (cascade)
POST	/api/studies/:id/blocks	Editor	Ajout d'un bloc
PATCH	/api/blocks/:id	Editor	Mise à jour bloc
DELETE	/api/blocks/:id	Editor	Suppression bloc
POST	/api/blocks/:id/questions	Editor	Ajout question
PATCH	/api/questions/:id	Editor	Mise à jour question
DELETE	/api/questions/:id	Editor	Suppression question
POST	/api/studies/:id/design	Editor	Configuration design expérimental
POST	/api/studies/:id/publish	Owner	Passage en statut COLLECTING

14.4 Passation participant·e (publique)

Méthode	Route	Auth	Description
GET	/api/run/:studyId	Public	Accès participant·e, init session
POST	/api/run/:studyId/responses	Public (sessionId)	Enregistrement réponse questionnaire
POST	/api/run/:studyId/stimulus	Public (sessionId)	Enregistrement réponse essai
POST	/api/run/:studyId/complete	Public (sessionId)	Clôture session, debriefing

14.5 Exports et administration

Méthode	Route	Auth	Description
GET	/api/studies/:id/export/csv	Editor	CSV (questions, format wide)
GET	/api/studies/:id/export/csv-trials	Editor	CSV (essais stimulus, RT)
GET	/api/studies/:id/export/xlsx	Editor	Excel multi-onglets

GET	/api/studies/:id/export/codebook	Editor	PDF (dictionnaire variables)
GET	/api/studies/:id/export/json	Editor	JSON (structure complète)
GET	/api/admin/stats	Admin	KPIs (utilisateur·rice·s, études, sessions)
GET	/api/admin/users	Admin	Liste paginée des utilisateur·rice·s
GET	/api/health	Public	Health check (statut, version, uptime)

15. Annexe B : Variables d'environnement (résumé)

La plateforme utilise un ensemble de variables d'environnement pour le backend, le frontend et la CI/CD. La liste exhaustive (noms exacts, valeurs par défaut, scope des secrets CI/CD) figure dans la version intégrale.

Au niveau résumé, on distingue :

- **Backend** : URL de connexion PostgreSQL, secret JWT, paramètres de port et de log, URL du frontend (CORS whitelist), clé API du fournisseur email, configuration des uploads.
- **Frontend** : URL du backend (compilée dans le bundle au build). Les variables préfixées `NEXT_PUBLIC_` sont publiques et ne doivent jamais contenir de secret.
- **CI/CD** : clés API Scaleway scopées au Container Registry et aux Containers, identifiants d'organisation et de containers, paramètres de build. Stockés dans GitHub Encrypted Secrets, accessibles uniquement aux jobs autorisés.

16. Annexe C : Glossaire

Terme	Définition
ACID	Atomicity, Consistency, Isolation, Durability. Propriétés des transactions PostgreSQL garantissant la cohérence des données.
AGPL	GNU Affero General Public License. Licence libre copyleft fort, exigeant la republication du code dérivé même pour des services en ligne.
Block	Unité atomique d'une étude MindCraft : Welcome, Instruction, Question, Stimulus/Task, Logic, Debriefing.
CDN	Content Delivery Network. Réseau d'edge servers cachés statiques près de l'utilisateur·rice final·e.
Codebook	Document PDF auto-généré listant chaque variable d'une étude. Équivalent du dictionnaire de données en sciences sociales.
CORS	Cross-Origin Resource Sharing. Mécanisme HTTP autorisant ou bloquant les requêtes entre origines différentes.
CSR	Client-Side Rendering. Rendu HTML par le navigateur via JavaScript.
CSRF	Cross-Site Request Forgery. Attaque où un site malveillant déclenche une action authentifiée à l'insu de l'utilisateur·rice.
Dependabot	Bot GitHub qui ouvre des Pull Requests automatiques pour mettre à jour les dépendances vulnérables ou obsolètes.
DKIM	DomainKeys Identified Mail. Authentification d'email par signature cryptographique du domaine expéditeur.
DMARC	Domain-based Message Authentication, Reporting & Conformance.
DOI	Digital Object Identifier. Identifiant pérenne pour publication ou logiciel.
DPA	Data Processing Agreement. Contrat de sous-traitance imposé par le RGPD.
ESM	ECMAScript Modules. Format de modules JavaScript natif (import/export).
FAIR	Findable, Accessible, Interoperable, Reusable. Principes pour la donnée et le logiciel de recherche.
HSTS	HTTP Strict Transport Security. Header qui force le navigateur à utiliser HTTPS.
JWT	JSON Web Token. Token compact signé permettant l'authentification sans état côté serveur.
Latin Square	Carré latin. Matrice utilisée pour le contrebalancement expérimental.
LSL	Lab Streaming Layer. Protocole standard en neurosciences pour synchroniser plusieurs flux de données physiologiques.
ORCID	Open Researcher and Contributor ID. Identifiant pérenne pour les chercheur·euse·s.
ORM	Object-Relational Mapping. Couche logicielle traduisant des objets dans le code en lignes en base relationnelle.
Prolific	Plateforme britannique de recrutement de participant·e·s académiques.
REST	Representational State Transfer. Style architectural d'API basée sur les ressources et les verbes HTTP.

RPO	Recovery Point Objective. Quantité maximale de données pouvant être perdue lors d'un incident.
RTO	Recovery Time Objective. Durée maximale d'interruption de service acceptée.
SemVer	Semantic Versioning. Convention de numérotation MAJOR.MINOR.PATCH.
SPDX	Software Package Data Exchange. Standard pour déclarer les licences dans le code source.
SPF	Sender Policy Framework. Authentification d'email par liste blanche d'IP autorisées.
SSG	Static Site Generation. Génération HTML au build.
SSR	Server-Side Rendering. Génération HTML côté serveur à chaque requête.
Stimulus	Élément présenté au·à la participant·e durant un essai (image, son, vidéo, texte).
SWHID	Software Heritage Identifier. Identifiant pérenne attribué par Software Heritage.
TOTP	Time-based One-Time Password. Algorithme de double authentification (RFC 6238).
Vitest	Framework de tests JavaScript rapide, compatible Jest, basé sur Vite.
Williams Design	Variante du carré latin équilibrant aussi les effets de carryover de premier ordre.
XSS	Cross-Site Scripting. Injection de JavaScript malveillant dans une page.
Zustand	Bibliothèque de gestion d'état React minimaliste.